

3 Kernel SVM

Consider the following kernel function defined over $z, z' \in \mathcal{Z}$: $k(z, z') = \begin{cases} 1 & \text{if } z = z', \\ 0 & \text{otherwise.} \end{cases}$

3.1 PSD [10 points]

Prove that for any integer $m > 0$, any $z_1, \dots, z_m \in \mathcal{Z}$, the $m \times m$ kernel matrix $K = [K_{ij}]$ is positive semi-definite, where $K_{ij} = k(z_i, z_j)$ for $i, j = 1, \dots, m$. (Let us assume that for $i \neq j$, we have $z_i \neq z_j$.) Hint: An $m \times m$ matrix K is positive semi-definite if $\forall v \in \mathbb{R}^m, v^T K v \geq 0$.

For $z_1, \dots, z_m \in \mathcal{Z}$ with $z_i \neq z_j$ for $i \neq j$, the kernel matrix $K \in \mathbb{R}^{m \times m}$ has entries

$$K_{ij} = k(z_i, z_j) = \begin{cases} 1, & i = j, \\ 0, & i \neq j, \end{cases}$$

so K is exactly the $m \times m$ identity matrix I_m .

To show K is positive semi-definite, take any $v \in \mathbb{R}^m$. Then

$$v^T K v = v^T I_m v = \sum_{i=1}^m v_i^2 \geq 0.$$

Since this holds for all $v \in \mathbb{R}^m$, the kernel matrix K is positive semi-definite.

3.2 Separability [10 points]

Given a training set $(z_1, y_1), \dots, (z_n, y_n)$ with binary labels, the dual SVM problem with the above kernel k will have parameters $\alpha_1, \dots, \alpha_n, b \in \mathbb{R}$. (Assume that for $i \neq j$, we have $z_i \neq z_j$.) The predictor for input z is

$$f(z) = \sum_{i=1}^n \alpha_i y_i k(z_i, z) + b.$$

Recall the prediction is $\text{sign}(f(z))$. Prove that there exists $\alpha_1, \dots, \alpha_n, b$ s.t. f correctly separates the training set. For a training set $(z_1, y_1), \dots, (z_n, y_n)$ with $y_i \in \{-1, +1\}$ and distinct z_i , the predictor is

$$f(z) = \sum_{i=1}^n \alpha_i y_i k(z_i, z) + b.$$

Because $k(z_i, z_j) = 1$ if $i = j$ and 0 otherwise, for any training point z_j we have

$$f(z_j) = \sum_{i=1}^n \alpha_i y_i k(z_i, z_j) + b = \alpha_j y_j + b.$$

We now choose explicit parameters. Let $\alpha_j = 1$ for all j and $b = 0$. Then, for each training example j ,

$$f(z_j) = 1 \cdot y_j = y_j,$$

and since $y_j \in \{-1, +1\}$,

$$\text{sign}(f(z_j)) = \text{sign}(y_j) = y_j.$$

Thus every training point is classified correctly.

Therefore there exist parameters $(\alpha_1, \dots, \alpha_n, b)$ (for example, $\alpha_i = 1$ for all i and $b = 0$) such that f correctly separates the training set.

explain 3.2 solution in detail

Let's go through **Section 3.2 (Separability)** carefully and unpack *why* the argument works step by step.

1. What kernel are we using?

The kernel is

$$k(z, z') = \begin{cases} 1 & \text{if } z = z' \\ 0 & \text{otherwise} \end{cases}$$

This is sometimes called a **Dirac / delta kernel**.

Its key property is:

- Each data point is **only similar to itself**
- Different points have **zero interaction**

You are also given the assumption that all training inputs are **distinct**:

$$z_i \neq z_j \quad \text{for } i \neq j$$

2. SVM predictor in the dual form

For a kernel SVM, the prediction function is

$$f(z) = \sum_{i=1}^n \alpha_i y_i k(z_i, z) + b$$

The final prediction is

$$\text{sign}(f(z))$$

Your goal is **not** to solve the optimization problem, but to **prove existence** of parameters $\alpha_1, \dots, \alpha_n, b$ that perfectly classify the training data.

3. Evaluate the predictor on a training point

Now evaluate $f(z)$ at one of the training points, say z_j :

$$f(z_j) = \sum_{i=1}^n \alpha_i y_i k(z_i, z_j) + b$$

Because of the kernel's definition:

- $k(z_i, z_j) = 1$ if $i = j$
- $k(z_i, z_j) = 0$ if $i \neq j$

So all terms vanish except one:

$$f(z_j) = \alpha_j y_j + b$$

This is the crucial simplification.

4. Key intuition

Each training point has **its own independent parameter** α_j .

There is:

- No interference from other points
- No coupling between examples

So the model can **set the output for each training point individually**.

This is why separability is guaranteed.

5. Explicit parameter choice

To prove existence, we just need to **exhibit one valid choice**.

Choose:

- $\alpha_j = 1$ for all j
- $b = 0$

Then for each training example z_j :

$$f(z_j) = 1 \cdot y_j + 0 = y_j$$

6. Correct classification

Since labels are binary:

$$y_j \in \{-1, +1\}$$

we get:

$$\text{sign}(f(z_j)) = \text{sign}(y_j) = y_j$$

So every training point is classified correctly.

7. Why this always works

This kernel effectively maps each data point to an **orthogonal basis vector** in feature space:

- No overlap
- No shared dimensions
- Maximum flexibility

This means:

- The training set is **always linearly separable**

- Even if labels are completely random

8. Big picture takeaway (exam-ready intuition)

- The delta kernel gives each point its **own private feature**
- The SVM can “memorize” the training data
- Separability is guaranteed as long as inputs are distinct
- This is a classic example of **perfect interpolation / overfitting**

One-line exam summary

Because $k(z_i, z_j) = \mathbf{1}[i = j]$, the SVM predictor at a training point reduces to $f(z_j) = \alpha_j y_j + b$. Choosing $\alpha_j = 1$ and $b = 0$ yields $f(z_j) = y_j$, so the training set is perfectly separated.

If you want, I can also:

- Interpret this in **feature-space geometry**
- Connect it to **nearest-neighbor behavior**
- Explain why this kernel generalizes terribly 🚩

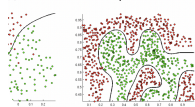


es: Gaussian/RBF

$$k(x, x') = \exp(-\gamma \|x - x'\|^2)$$

ernels, we are projecting to an infinite dimensi

.0 $\gamma = 100$ $\gamma =$



role of the gamma parameter in SVMs with an RBF kernel?

ols the trade-off between maximizing the margin and minimizing clas

ols the shape of the decision boundary

ols the width of the RBF kernel

ols the complexity of the decision boundary

C. It controls the width of the RBF kernel

VMs with an RBF kernel, the gamma parameter controls the width of

as how close a data point must be to a support vector to influence the c

why is C correct when in the images I can see that it controls the complexity of the decision boundary

This is a **really good question**, and your intuition from the images is actually *right*. The confusion comes from the difference between:

- what γ *directly* controls (formal definition)
- vs.
- what γ *induces* as a downstream effect (visual/behavioral outcome)

Let's make that distinction very explicit.

1. What γ controls *mathematically* (the exam-correct answer)

The RBF kernel is

$$k(x, x') = \exp \left(- \gamma \|x - x'\|^2 \right)$$

Here, γ appears only in one place:

it scales the squared distance inside the exponential.

That means:

- Large $\gamma \rightarrow$ distance is magnified $\rightarrow$ kernel decays **very fast**
- Small $\gamma \rightarrow$ distance is shrunk $\rightarrow$ kernel decays **slowly**

So **formally and directly**:

γ controls the width (spread) of the RBF kernel

This is why **C** is the correct answer.

Sometimes this is written as

$$\gamma = \frac{1}{2\sigma^2}$$

so γ is literally the inverse of the Gaussian variance.

2. What γ controls *geometrically* (what you see in the plots)

Now let's connect this to the images you showed.

Small γ (wide kernel)

- Each point influences a **large region**
- Many points overlap in influence

- Decision boundary is **smooth**
- Model has **low effective complexity**

Large γ (narrow kernel)

- Each point influences **only its immediate neighborhood**
- Points act almost independently
- Decision boundary becomes **wiggly and fragmented**
- Model has **high effective complexity**

So visually:

increasing $\gamma \rightarrow$ more complex decision boundary

This is exactly what your plots show.

3. Why option D feels right but is marked wrong

Why D feels correct

Because it is **empirically and intuitively true**:

γ affects how complex the learned boundary can become

You are *not wrong* to think this.

Why D is marked incorrect

Because it is **imprecise and indirect**.

- γ does **not** directly regularize complexity
- That role belongs to **C (the margin penalty)**

γ only controls **locality of similarity**.

Complexity is a **consequence**, not the definition.

Exams (especially ML theory exams) care about:

what parameter controls what in the objective/kernel definition

4. Relationship between γ and complexity (important nuance)

A very clean way to think about it:

γ controls the kernel width $\rightarrow$ kernel width controls locality $\rightarrow$ locality controls boundary complexity

So the causal chain is:

$\gamma \rightarrow \text{kernel width} \rightarrow \text{decision boundary shape/complexity}$

Option **C** names the first link,

Option **D** names the last link.

Only **C** is strictly correct.

5. Contrast with C (the regularization parameter)

To really lock this in:

- **C** controls:
 - margin vs training error
 - explicit regularization
- **γ** controls:
 - similarity decay
 - spatial resolution of the feature space

That's why textbooks often say:

C controls regularization, γ controls locality

6. Exam-ready takeaway (this is the key sentence)

γ does **not directly** control model complexity; it controls the **width of the RBF kernel**, which **indirectly affects** how complex the decision boundary can become.

7. One-line answer you can safely write on an exam

The γ parameter determines the width of the RBF kernel, i.e., how quickly similarity decays with distance; smaller γ yields smoother decision boundaries, while larger γ

allows more localized and complex boundaries.

If you want, I can also:

- Explain how **y** and **C** interact
- Show a **failure mode** when both are large
- Tie this to **bias–variance tradeoff** formally



is the boundary a non linear function? do we have margins?

Yes to both — but with **important clarifications**. Let's be precise, in an exam-safe way.

1. Is the decision boundary nonlinear?

In input space (x-space): YES

With an RBF kernel, the decision boundary in the original input space is **nonlinear**.

Why?

The decision function is

$$f(x) = \sum_{i=1}^n \alpha_i y_i \exp(-\gamma \|x - x_i\|^2) + b$$

This is a **sum of Gaussians**, which is **not linear in x** .

So in the input space, the boundary can be curved, wiggly, or highly complex.

This is exactly what you see in the plots.

In feature space (RKHS): NO

In the (implicit) feature space induced by the kernel:

$$f(x) = \langle w, \phi(x) \rangle + b$$

This is **linear**.

So the correct statement is:

The SVM finds a **linear separator in feature space**, which corresponds to a **nonlinear decision boundary in input space**.

2. Do we still have margins with an RBF kernel?

Short answer: YES

Margins absolutely still exist.

3. What “margin” means with kernels

The margin is **not defined in input space**.

It is defined in **feature space**:

$$\text{margin} = \frac{1}{\|w\|}$$

Even though the feature space is:

- infinite-dimensional (for RBF)
- not explicitly constructed

the margin is still **well-defined mathematically**.

4. How margin maximization works with RBF kernels

The SVM optimization problem is still:

$$\min_{w, b, \xi} \quad \frac{1}{2} \|w\|^2 + C \sum_i \xi_i$$

subject to:

$$y_i(\langle w, \phi(x_i) \rangle + b) \geq 1 - \xi_i$$

Nothing about this changes when using kernels.

So:

- The classifier still **maximizes margin**
 - Support vectors are still the points **closest to the boundary in feature space**
-

5. Why margins are hard to “see” in RBF plots

In linear SVMs:

- Margin corresponds to **parallel lines** in input space
- Easy to visualize

In RBF SVMs:

- Margin lives in feature space
- Input-space projection is **warped and nonlinear**
- Margin width in x-space is **not constant**

So you don’t see “parallel margins” in the plots — but they **exist**.

6. Important subtlety (this is where people get confused)

With large γ :

- Feature space becomes extremely high-resolution
- Points become almost orthogonal
- Margin in feature space can be large
- But boundary in input space can look extremely wiggly

This is **not a contradiction**.

A large margin in feature space does **not** imply a smooth boundary in input space.

7. Final exam-ready summary

Yes, with an RBF kernel the decision boundary is nonlinear in the input space but linear in the implicit feature space.

Yes, SVMs with RBF kernels still maximize a margin, which is defined in feature space (RKHS), not in the original input space.

If you want, I can also explain:

- how γ affects **margin geometry**
- why "large margin $\neq$ simple boundary"
- how support vectors behave as $\gamma \rightarrow \infty$

